



# Cours Pratique : Authentification Complète (Backend Node.js & Frontend React)

Ce document explique de A à Z comment nous avons mis en place un système d'authentification complet (Inscription & Connexion) pour l'application **CREATOR**.

Il lie un backend (Node.js/Express/MongoDB) à un frontend (React) en utilisant des standards de sécurité modernes comme le hachage de mot de passe et les Tokens JWT.



## 1. Les Concepts Clés

Avant de coder, voici les concepts fondamentaux à comprendre :

1. **API REST (Backend)** : C'est le "serveur". Il écoute les requêtes du Frontend, interagit avec la base de données (MongoDB), et renvoie une réponse.
2. **Mongoose** : C'est un outil qui permet à notre code Node.js de parler facilement à notre base de données MongoDB en utilisant des "Modèles" (Schémas).
3. **Bcrypt** : Une librairie de sécurité. **On ne sauvegarde jamais un mot de passe en clair !** Bcrypt transforme "monmotdepasse123" en une chaîne illisible comme `$2a$10$X...` (le hachage).
4. **JWT (JSON Web Token)** : C'est une sorte de "badge visiteur" numérique. Quand l'utilisateur se connecte avec succès, le serveur lui donne un Token. Le Frontend gardera ce Token (dans le `localStorage`) et le montrera au serveur à chaque fois qu'il voudra accéder à une page protégée.



## 2. Étape 1 : Création des Modèles (Base de Données)

Dans MongoDB, les données sont structurées par des Schémas Mongoose. Nous avons créé deux modèles principaux : `Creator` et `Entreprise`.

**Fichier : `backend-creator/models/Creator.js`** Nous avons défini les informations nécessaires (Nom, Prénom, Email, Mot de passe). *La Bonne Pratique* : L'email doit être défini comme `unique: true` pour éviter les doublons. L'option `timestamps: true` à la fin du schéma permet à Mongoose de gérer automatiquement les dates de création ( `createdAt` ) et de modification ( `updatedAt` ).



## 3. Étape 2 : Le Contrôleur d'Authentification (La Logique)

Le cœur de l'authentification se trouve dans le "Controller". C'est lui qui fait le travail quand une requête arrive.

**Fichier : `backend-creator/controllers/authController.js`**

### L'Inscription (Register)

1. **Vérification** : On regarde si l'email existe déjà dans la base.
2. **Sécurité** : On utilise `bcrypt.hash()` pour crypter le mot de passe.
3. **Sauvegarde** : On enregistre le nouvel utilisateur dans MongoDB.

### La Connexion (Login)

1. **Recherche** : On cherche l'utilisateur via son email (on vérifie dans `Creator` puis dans `Entreprise`).

2. **Comparaison** : On utilise `bcrypt.compare()` pour voir si le mot de passe tapé correspond au mot de passe crypté en base de données.
  3. **Création du Badge (JWT)** : Si tout est bon, on utilise `jwt.sign()` pour générer un Token sécurisé. Ce Token contient l'ID de l'utilisateur de manière cryptée.
- 

## 4. Étape 3 : Les Routes et le Middleware de Sécurité

Les routes sont les "URL" sur lesquelles le Frontend va envoyer ses requêtes.

Fichiers : `routes/authRoutes.js` et `routes/userRoutes.js`

- `POST /api/auth/register-creator`
- `POST /api/auth/login`

**Le Middleware (Le Vigile)** : `middleware/authMiddleware.js` C'est une fonction appelée `protect()`. Avant d'autoriser l'accès à une route sensible (comme voir son profil), cette fonction :

1. Cherche le Token JWT dans l'en-tête de la requête ( `Headers: Authorization: Bearer <token>` ).
  2. Vérifie si le Token est valide (pas expiré, pas falsifié) grâce à `jwt.verify()`.
  3. Si c'est valide, elle laisse passer l'utilisateur. Sinon, elle renvoie une erreur `401 Unauthorized`.
- 

## 5. Étape 4 : L'intégration Frontend (React)

Côté interface, nous utilisons `axios` pour envoyer nos requêtes au backend, et `react-router-dom` pour naviguer entre les pages.

### La Page de Connexion ( `Login.jsx` )

Quand l'utilisateur remplit le formulaire et clique sur "Se connecter", on envoie une requête `POST` avec son email et mot de passe à `/api/auth/login`. **L'action magique** : Le backend nous renvoie le fameux Token JWT. On le sauvegarde immédiatement dans le navigateur avec `localStorage.setItem('token', token)`. C'est ça qui "maintient" la connexion !

### La Page Sécurisée ( `Dashboard.jsx` )

Quand l'utilisateur arrive sur son espace privé, la page vérifie s'il possède un Token dans son `localStorage`.

- **S'il n'en a pas**, on le renvoie à la page de Login (Sécurité !).
- **S'il en a un**, on fait une requête `GET` à `/api/users/profile` en **attachant le Token** dans les en-têtes (Headers) de la requête. Le vigile du backend ( `protect()` ) verra le token, l'acceptera, et renverra les informations du profil pour les afficher à l'écran.

### Se déconnecter

C'est extrêmement simple : il suffit de supprimer le "badge" ! On appelle `localStorage.removeItem('token')` et on redirige vers la page de connexion.

---

## Conclusion pour les Étudiants

L'authentification peut faire peur au début, mais elle suit toujours la même logique :

1. **Je m'inscris** : Mon mot de passe est crypté et stocké en sécurité.
2. **Je me connecte** : Je prouve mon identité, le serveur me donne un *Token* (Badge).
3. **Je navigue** : Je présente mon *Token* à chaque fois que je veux accéder à une zone VIP (Route protégée).
4. **Je me déconnecte** : Je jette mon *Token*.

 **Vous êtes maintenant capables de créer un système de connexion professionnel !**